Google Cloud H2S

Gen AI Academy

APAC Edition

Build in APAC. Build for the world.

Participant Details

Participant Name: Adityo Nugroho

Problem Statement: Build a multi-agent AI system that helps users manage tasks, schedules, and information by interacting with multiple tools and data sources.

Brief about the idea

NetPulse AI - TelecomOps Multi-Agent Assistant

NetPulse AI automates the first-response workflow for telecom Network Operations Center (NOC) teams. When a customer complaint arrives, the system:

1. Classifies the issue (network, billing, hardware, service, or general)
2. Investigates live network events from BigQuery via MCP Toolbox
3. Analyzes Call Detail Records (CDR) from AlloyDB
4. Generates and persists a structured incident ticket with NOC recommendations

The entire 4-step workflow completes in 25-30 seconds, replacing what previously required a trained operator to manually query 3 separate systems over 10-30 minutes.

Your solution should be able to explain the following:

- How did you approach the common problem statement and translate it into a working solution using ADK, MCP/AlloyDB AI while ensuring Hackathon's core requirements are met?
- What real-world problem does your solution address, and what practical impact does it create for users?
- What is the core approach or workflow behind your solution?

Approach & Core Requirements

- Built with Google ADK 1.14 SequentialAgent orchestrating 4 LlmAgent sub-agents.
- Each sub-agent has dedicated tools and writes results to session state via output_key, feeding downstream agents.
- Integrated MCP Toolbox for Databases as the BigQuery bridge (direct BigQuery MCP fails on Cloud Run).
- AlloyDB queried via native SQLAlchemy + pg8000 tools.
- Deployed on Cloud Run; Vertex AI inference on asia-southeast1 to minimize APAC latency and Dynamic Shared Quota contention.

Real-World Problem

Telecom NOC teams manually query network management systems, CDR databases, and ticketing systems for every complaint which is a slow, error-prone process. NetPulse compresses this to under 30 seconds by automatically correlating evidence from two independent data sources.

Core Workflow

User complaint → Classifier → Network Investigator (BigQuery/MCP) → CDR Analyzer (AlloyDB) → Response Formatter (ticket)

Opportunities

- How different is it from any of the other existing ideas?
- USP of the proposed solution

What makes NetPulse AI different:

- Truly multi-agent, not multi-tool: 4 specialized sub-agents each own a distinct responsibility. A monolithic agent with all tools would lose the structured handoff and output_key state chain.
- Cross-source evidence correlation: Automatically links BigQuery network events with AlloyDB CDR data to surface root causes. No human joining of sources required.
- Persistent structured output: Every run creates an auditable incident ticket in AlloyDB, not just a chat response. Tickets are queryable and feed downstream NOC workflows.
- APAC-optimized deployment: Vertex AI inference on asia-southeast1 for reduced latency for Indonesian/APAC users; DSQ contention avoided by staying off us-central1.
- Transparent reasoning: ADK Trace tab exposes every LLM call, tool invocation, and state value.

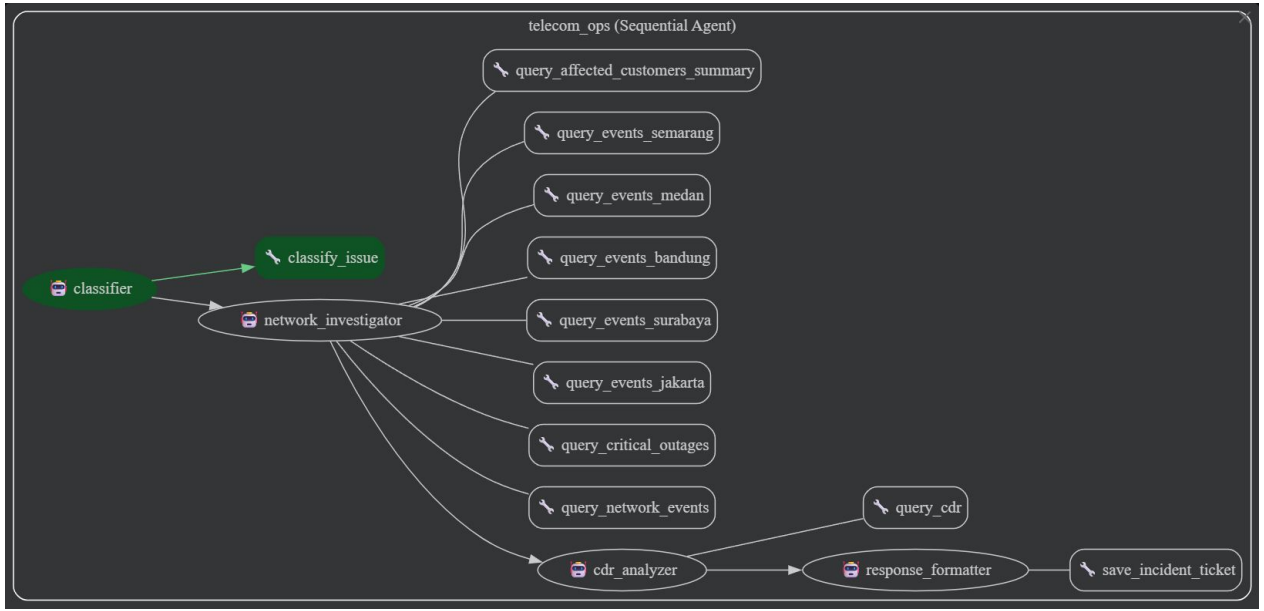
List of features offered by the solution

Core Features

- Automatic complaint classification into 5 categories: network, billing, hardware, service, general
- Region detection for 5 Indonesian cities: Jakarta, Surabaya, Bandung, Medan, Semarang
- Network event investigation via MCP Toolbox: 8 BigQuery tools (5 per-region + outages + customers + summary)
- CDR analysis via AlloyDB call_records: parameterized by region and call_status
- Incident ticket generation with NOC recommendation, persisted to AlloyDB incident_tickets table
- Session state chaining: each sub-agent's output feeds downstream agents via ADK output_key
- Boot-resilient: MCP Toolbox wrapped in try/except; service starts even if toolbox is cold
- **32 incident tickets created across 5 Indonesian regions and 3 issue categories during pre-submission stress testing**
- Live Cloud Run deployment with a custom NetPulse UI (Flask + SSE chat) v1 and ADK Dev UI (Events + Trace tabs)
  - **NetPulse UI:** https://netpulse-ui-486319900424.us-central1.run.app
  - **ADK Dev UI:** https://telecom-ops-assistant-486319900424.us-central1.run.app

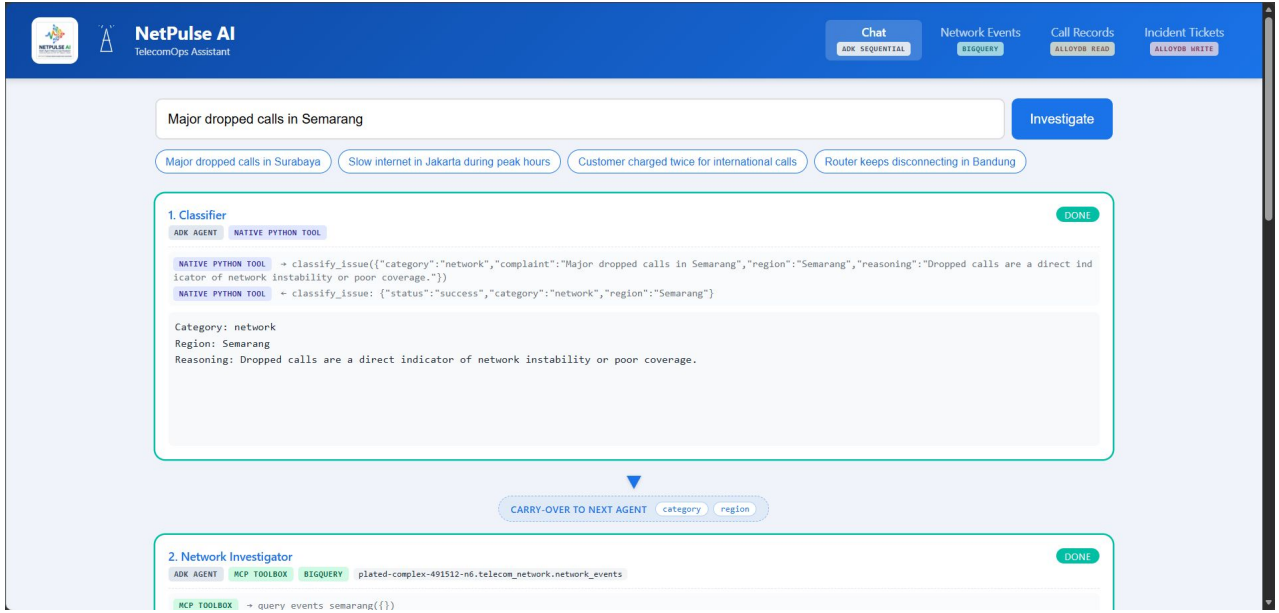
Process flow diagram or Use-case diagram

telecom_ops is the NetPulse AI Google ADK SequentialAgent that orchestrates four LlmAgent sub-agents in fixed order: (1) Classifier turns the natural-language complaint into structured category and region; (2) Network Investigator queries BigQuery via MCP Toolbox for events relevant to that region; (3) CDR Analyzer pulls matching call records from AlloyDB; (4) Response Formatter writes a structured ticket to AlloyDB and emits the final report. Each agent's output_key feeds the next via session.state.



Wireframes/Mock diagrams of the proposed solution

The NetPulse AI UI chat tab is the primary user surface. A free-text complaint sits at the top, four agent pipeline cards (Classifier, Network Investigator, CDR Analyzer, and Response Formatter) run below with carry-over labels showing the session.state keys passed forward, and a final dark Incident Report card sits at the bottom. Each card streams its tool calls and text output via Server-Sent Events as the ADK SequentialAgent runs.



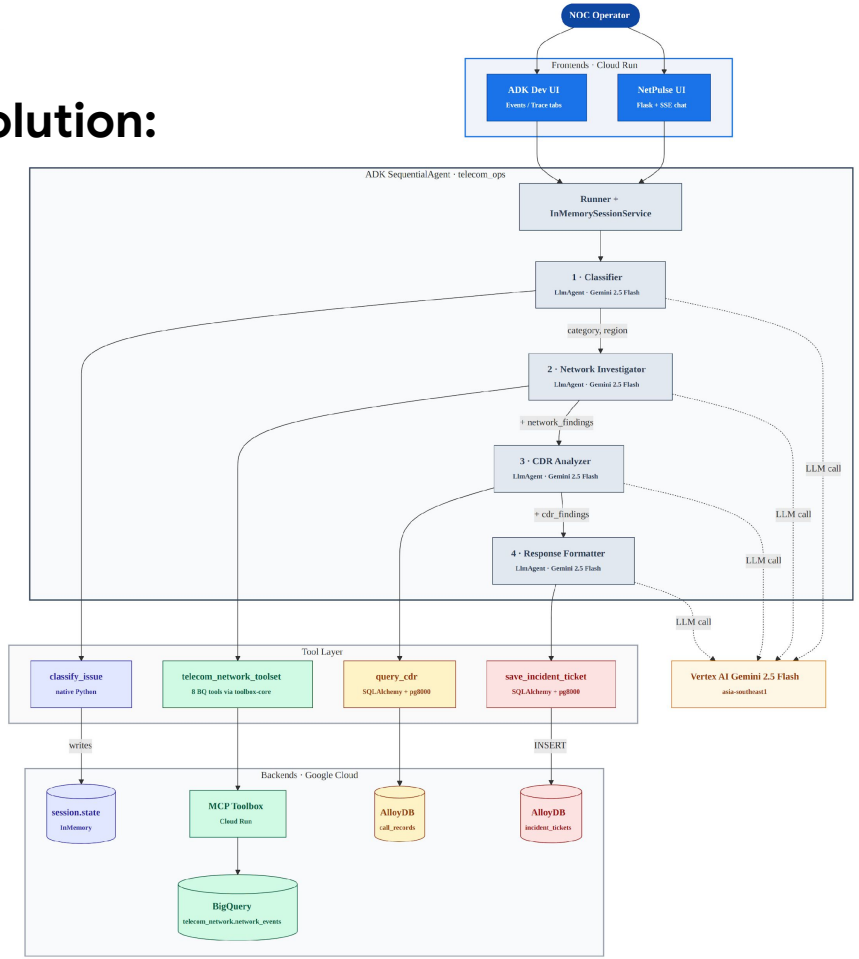
Architecture diagram of the proposed solution:

End-to-end architecture:

NOC operator hits one of two Cloud Run frontends, the NetPulse UI Flask app or the ADK Dev UI. Both call into the same ADK Runner and InMemorySessionService driving the telecom_ops SequentialAgent. Each LlmAgent has a dedicated tool.

The classifier calls classify_issue (native Python), the network investigator calls telecom_network_toolset which routes through MCP Toolbox to BigQuery, the CDR analyzer calls query_cdr against AlloyDB, and the response formatter calls save_incident_ticket to write back into AlloyDB.

All four LLM calls go to Vertex AI Gemini 2.5 Flash in asia-southeast1 to minimize APAC latency and DSQ contention.



Technologies/Google Services used in the solution

(Why did you choose your specific AI stack and system design, and how does it support scalability and real-world deployment?)

Google ADK 1.14.0
SequentialAgent + LlmAgent orchestration. Built-in state management, output_key chaining, and before_model_callback for pacing. Uses native Python with zero framework code.

Gemini 2.5 Flash on Vertex AI (asia-southeast1)
Best latency/cost tradeoff for APAC. Avoids Dynamic Shared Quota throttling on us-central1 which is heavily contested during the hackathon period.

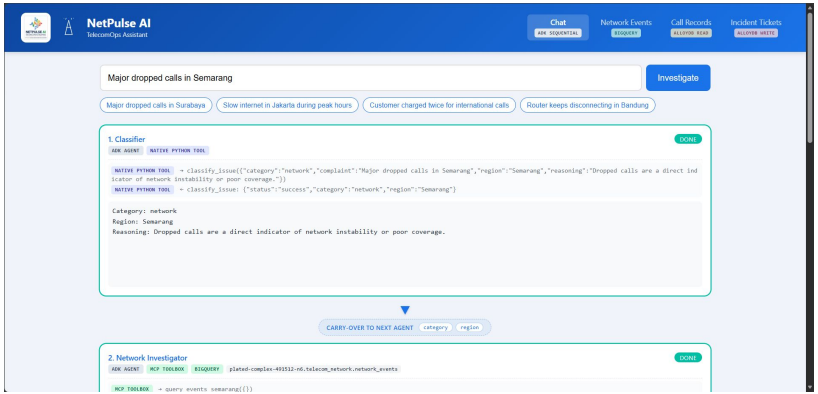
MCP Toolbox for Databases (Cloud Run)
Direct BigQuery MCP over HTTP fails on Cloud Run with 403 errors. Toolbox is the proven ADK-compatible bridge exposing 8 SQL tools over the BigQuery network_events dataset.

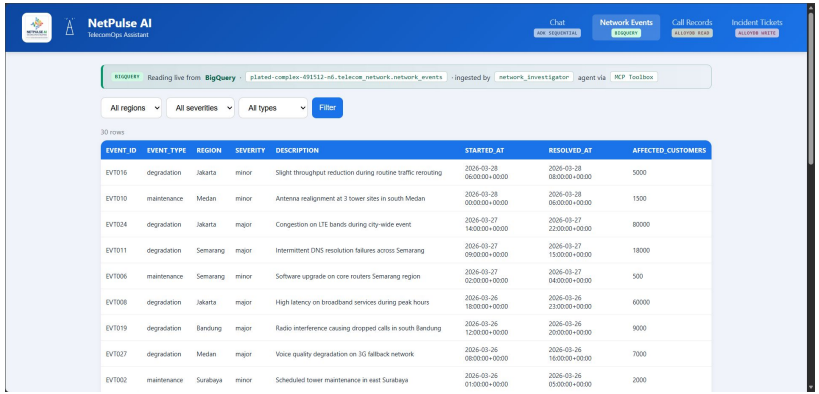
AlloyDB for PostgreSQL
Hosts call_records (50 rows, CDR data) and incident_tickets (persistent output). Public IP for local dev; private VPC IP for Cloud Run via AlloyDB connector.

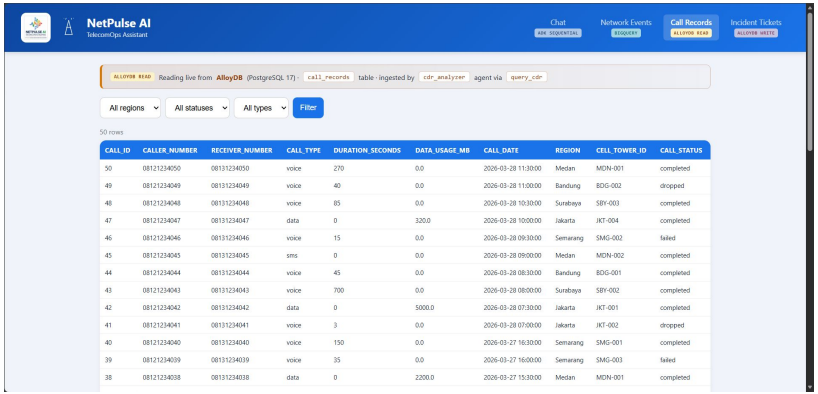
BigQuery (telecom_network.network_events)
30-row dataset covering 5 Indonesian cities. Pre-populated with outages, maintenance, and restoration events, providing a realistic NOC investigation scenario.

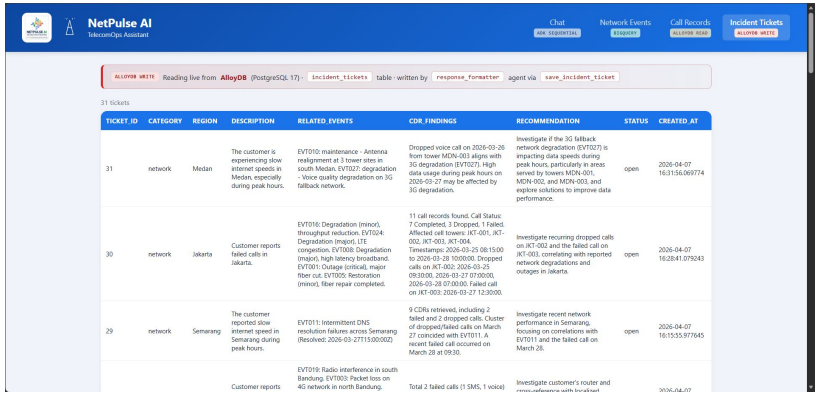
Snapshots of the prototype

NetPulse AI Custom UI
Top row: Chat (ADK SequentialAgent), Network Events (BigQuery)
Bottom row: Call Records (AlloyDB read), Incident Tickets (AlloyDB write)



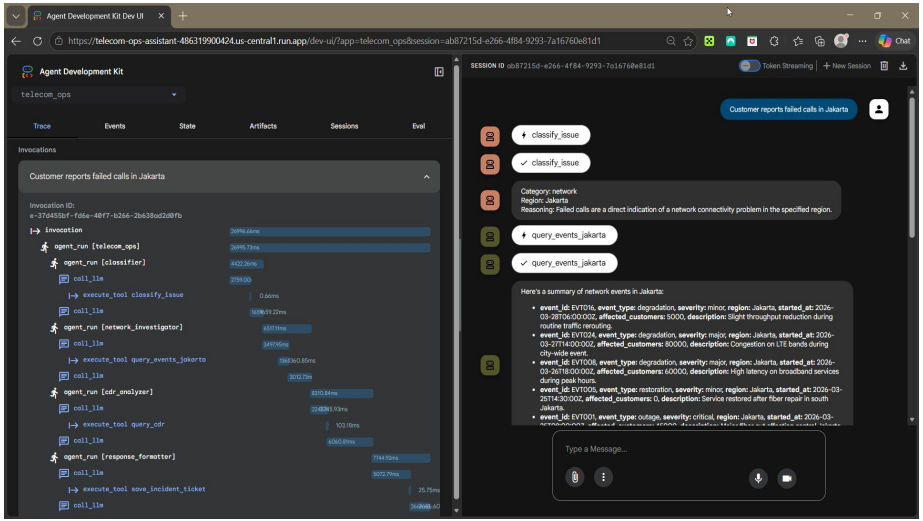


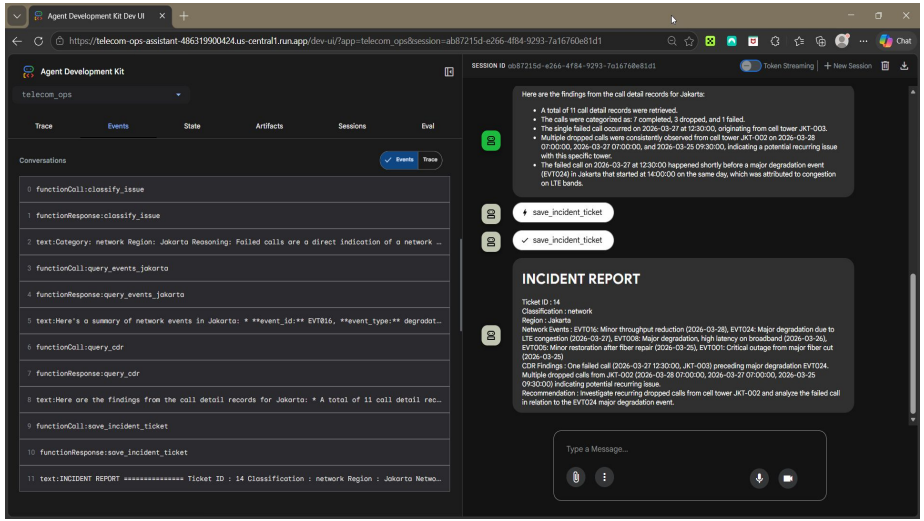


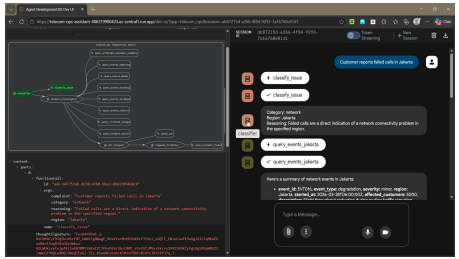


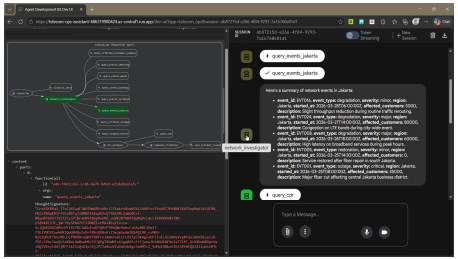
Snapshots of the prototype

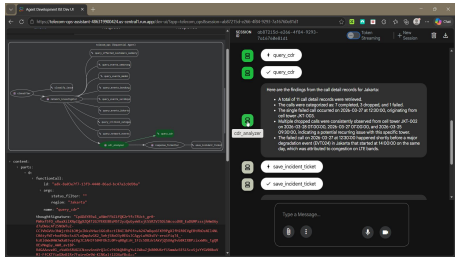
Top row: the ADK Dev UI Trace tab on the left shows the full tool-invocation timeline for one query, and the Events tab on the right shows the streaming sub-agent conversation.
Bottom row: the four sub-agents Classifier, Network Investigator, CDR Analyzer, and Response Formatter, each captured with their tool calls and final output.

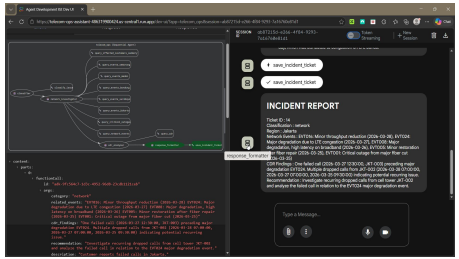












Google Cloud H2S

Gen AI Academy APAC Edition

Build in APAC. Build for the world.

Thank you